

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

*I _B.Poorvaja(192473008)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

		address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1.SCHOOL MANAGEMENT SYSTEM :

Relations :

Student(StudentID, StudentName, ClassID)

Class(ClassID, ClassName, TeacherID)

Teacher(TeacherID, TeacherName)

SQL Query :

```
CREATE DATABASE schooldb;
```

```
USE schooldb;
```

```
SELECT DATABASE();
```

```
CREATE TABLE Teacher (
```

```
    TeacherID INT PRIMARY KEY,
```

```
    TeacherName VARCHAR(50)
```

```
);
```

```
CREATE TABLE Class (
```

```
    ClassID INT PRIMARY KEY,
```

```
    ClassName VARCHAR(20),
```

```
    TeacherID INT,
```

```
    FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID)
```

```
);
```

```
CREATE TABLE Student (
```

```
    StudentID INT PRIMARY KEY,
```

```
    StudentName VARCHAR(50),
```

```
    ClassID INT,
```

```
    FOREIGN KEY (ClassID) REFERENCES Class(ClassID)
```

```
);
```

```
INSERT INTO Teacher VALUES
```

```
(1,'Ramesh'),
```

```
(2,'Priya');
```

```
INSERT INTO Class VALUES
```

```
(101,'10A',1),
```

```
(102,'10B',2);
```

```

INSERT INTO Student VALUES
(1,'Rahul',101),
(2,'Sneha',102),
(3,'Amit',101);
SELECT
    s.StudentID,
    s.StudentName,
    c.ClassName,
    t.TeacherName AS ClassTeacher
FROM Student s
JOIN Class c
ON s.ClassID = c.ClassID
JOIN Teacher t
ON c.TeacherID = t.TeacherID;

```

```

mysql> USE schooldb;
Database changed
mysql> SELECT DATABASE();
+-----+
| DATABASE() |
+-----+
| schooldb   |
+-----+
1 row in set (0.00 sec)

```

```

mysql> select* from Teacher;
+-----+-----+
| TeacherID | TeacherName |
+-----+-----+
| 1         | Ramesh      |
| 2         | Priya       |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Class;
+-----+-----+-----+
| ClassID | ClassName | TeacherID |
+-----+-----+-----+
| 101     | 10A       | 1         |
| 102     | 10B       | 2         |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | StudentName | ClassID |
+-----+-----+-----+
| 1         | Rahul       | 101     |
| 2         | Sneha       | 102     |
| 3         | Amit        | 101     |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> INSERT INTO Student VALUES
-> (1,'Rahul',101),
-> (2,'Sneha',102),
-> (3,'Amit',101);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT
-> s.StudentID,
-> s.StudentName,
-> c.ClassName,
-> t.TeacherName AS ClassTeacher
-> FROM Student s
-> JOIN Class c
-> ON s.ClassID = c.ClassID
-> JOIN Teacher t
-> ON c.TeacherID = t.TeacherID;
+-----+-----+-----+-----+
| StudentID | StudentName | ClassName | ClassTeacher |
+-----+-----+-----+-----+
| 1         | Rahul       | 10A       | Ramesh       |
| 3         | Amit        | 10A       | Ramesh       |
| 2         | Sneha       | 10B       | Priya        |
+-----+-----+-----+-----+
3 rows in set (0.01 sec)

```

2. HOSPITAL APPOINTMENT SYSTEM

Relational Schema :

Patient(PatientID, PatientName)

Doctor(DoctorID, DoctorName, Specialization)

Appointment(AppointmentID, PatientID, DoctorID, AppointmentDate)

SQL Query :

```
CREATE DATABASE hospitaldb;
SHOW DATABASES;
USE hospitaldb;
SELECT DATABASE();
CREATE TABLE Doctor
(
    DoctorID INT PRIMARY KEY,
    DoctorName VARCHAR(50),
    Specialization VARCHAR(50)
);
SHOW TABLES;
DESC Doctor;
CREATE TABLE Patient
(
    PatientID INT PRIMARY KEY,
    PatientName VARCHAR(50),
    Age INT
);
SHOW TABLES;
DESC Patient;
CREATE TABLE Appointment
(
    AppointmentID INT PRIMARY KEY,
    PatientID INT,
    DoctorID INT,
    AppointmentDate DATE,
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);
SHOW TABLES;
DESC Appointment;
INSERT INTO Doctor VALUES
(1,'Dr. Ravi','Cardiology'),
(2,'Dr. Priya','Dermatology');
SELECT * FROM Doctor;

INSERT INTO Patient VALUES
(101,'Rahul',20),
(102,'Sneha',21),
(103,'Amit',22);
SELECT * FROM Patient;
INSERT INTO Appointment VALUES
(1,101,1,'2026-08-01'),
```

```
(2,102,1,'2026-08-02'),
(3,103,1,'2026-08-03'),
(4,101,1,'2026-08-04'),
(5,102,1,'2026-08-05'),
(6,103,1,'2026-08-06'),
(7,101,1,'2026-08-07'),
(8,102,1,'2026-08-08'),
(9,103,1,'2026-08-09'),
(10,101,1,'2026-08-10'),
(11,102,1,'2026-08-11'),
(12,103,2,'2026-08-12');
SELECT * FROM Appointment;
```

```
SELECT
    d.DoctorID,
    d.DoctorName,
    COUNT(a.AppointmentID) AS TotalAppointments
FROM Doctor d
JOIN Appointment a
ON d.DoctorID = a.DoctorID
WHERE MONTH(a.AppointmentDate) = MONTH(CURDATE())
AND YEAR(a.AppointmentDate) = YEAR(CURDATE())
GROUP BY d.DoctorID, d.DoctorName
HAVING COUNT(a.AppointmentID) > 10;
```

```
mysql> USE hospitaldb;
Database changed
mysql> SELECT DATABASE();
+-----+
| DATABASE() |
+-----+
| hospitaldb |
+-----+
```

```
mysql> SELECT * FROM Doctor;
+-----+-----+-----+
| DoctorID | DoctorName | Specialization |
+-----+-----+-----+
| 1 | Dr. Ravi | Cardiology |
| 2 | Dr. Priya | Dermatology |
+-----+-----+-----+
```

```
mysql> SELECT * FROM Patient;
+-----+-----+-----+
| PatientID | PatientName | Age |
+-----+-----+-----+
| 101 | Rahul | 20 |
| 102 | Sneha | 21 |
| 103 | Amit | 22 |
+-----+-----+-----+
```

```
mysql> SELECT * FROM Appointment;
+-----+-----+-----+-----+
| AppointmentID | PatientID | DoctorID | AppointmentDate |
+-----+-----+-----+-----+
| 1 | 101 | 1 | 2026-08-01 |
| 2 | 102 | 1 | 2026-08-02 |
| 3 | 103 | 1 | 2026-08-03 |
| 4 | 101 | 1 | 2026-08-04 |
| 5 | 102 | 1 | 2026-08-05 |
| 6 | 103 | 1 | 2026-08-06 |
| 7 | 101 | 1 | 2026-08-07 |
| 8 | 102 | 1 | 2026-08-08 |
| 9 | 103 | 1 | 2026-08-09 |
| 10 | 101 | 1 | 2026-08-10 |
| 11 | 102 | 1 | 2026-08-11 |
| 12 | 103 | 2 | 2026-08-12 |
+-----+-----+-----+-----+
```

```
mysql> SELECT * FROM Appointment;
+-----+-----+-----+-----+
| AppointmentID | PatientID | DoctorID | AppointmentDate |
+-----+-----+-----+-----+
| 1 | 101 | 1 | 2026-08-01 |
| 2 | 102 | 1 | 2026-08-02 |
| 3 | 103 | 1 | 2026-08-03 |
| 4 | 101 | 1 | 2026-08-04 |
| 5 | 102 | 1 | 2026-08-05 |
| 6 | 103 | 1 | 2026-08-06 |
| 7 | 101 | 1 | 2026-08-07 |
| 8 | 102 | 1 | 2026-08-08 |
| 9 | 103 | 1 | 2026-08-09 |
| 10 | 101 | 1 | 2026-08-10 |
| 11 | 102 | 1 | 2026-08-11 |
| 12 | 103 | 2 | 2026-08-12 |
+-----+-----+-----+-----+
```

3. E-COMMERCE SYSTEM

Tables

Product(ProductID, ProductName)

Orders(OrderID, ProductID)

SQL QUERY :

```
CREATE DATABASE ecommerce;
SHOW DATABASES;
USE ecommerce;
SELECT DATABASE();
CREATE TABLE Product
(
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Price DECIMAL(10,2)
);
SHOW TABLES;
DESC Product;
CREATE TABLE Orders
(
    OrderID INT PRIMARY KEY,
    ProductID INT,
    Quantity INT,
    OrderDate DATE,
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
SHOW TABLES;
DESC Orders;
INSERT INTO Product VALUES
(101,'Laptop',55000),
(102,'Mouse',500),
(103,'Keyboard',1200),
(104,'Monitor',15000),
(105,'Printer',8000);
SELECT * FROM Product;
INSERT INTO Orders VALUES
(1,101,2,'2026-08-01'),
(2,102,5,'2026-08-02'),
(3,103,3,'2026-08-03');
SELECT * FROM Orders;
SELECT
    ProductID,
    ProductName
FROM Product
WHERE ProductID NOT IN
(
    SELECT ProductID
    FROM Orders
);
```

```
mysql> SELECT * FROM Product;
```

ProductID	ProductName	Price
101	Laptop	55000.00
102	Mouse	500.00
103	Keyboard	1200.00
104	Monitor	15000.00
105	Printer	8000.00

```
mysql> SELECT * FROM Orders;
```

OrderID	ProductID	Quantity	OrderDate
1	101	2	2026-08-01
2	102	5	2026-08-02
3	103	3	2026-08-03

```
-> );
```

ProductID	ProductName
104	Monitor
105	Printer

4. UNIVERSITY DATABASE

Relational Algebra

DBMS $\leftarrow \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='DBMS'}$

(Enroll $\bowtie$ Course))

OS $\leftarrow \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='OS'}$

(Enroll $\bowtie$ Course))

DBMS $\cap$ OS

SQL QUERY:

```
mysql> USE universitydb;
Database changed
mysql> SELECT DATABASE();
+-----+
| DATABASE() |
+-----+
| universitydb |
+-----+
1 row in set (0.00 sec)

mysql> CREATE TABLE Student
-> (
-> StudentID INT PRIMARY KEY,
-> StudentName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Course
-> (
-> CourseID INT PRIMARY KEY,
-> CourseName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Enroll
-> (
-> StudentID INT,
-> CourseID INT,
```

```
-> PRIMARY KEY(StudentID, CourseID),
-> FOREIGN KEY(StudentID) REFERENCES Student(StudentID),
-> FOREIGN KEY(CourseID) REFERENCES Course(CourseID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student VALUES
-> (1,'Rahul'),
-> (2,'Sneha'),
-> (3,'Amit');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Enroll VALUES
-> (1,101),
-> (1,102),
-> (2,101),
-> (3,102),
-> (3,103);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Course VALUES
-> (101,'DBMS'),
-> (102,'OS'),
-> (103,'Python');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT s.StudentID, s.StudentName
-> FROM Student s
-> JOIN Enroll e1 ON s.StudentID = e1.StudentID
-> JOIN Course c1 ON e1.CourseID = c1.CourseID
-> JOIN Enroll e2 ON s.StudentID = e2.StudentID
-> JOIN Course c2 ON e2.CourseID = c2.CourseID
-> WHERE c1.CourseName = 'DBMS'
-> AND c2.CourseName = 'OS';
```

OUTPUT:

StudentID	StudentName
1	Rahul

5. Payroll System

SQL Query:

```
CREATE DATABASE payrolldb;
USE payrolldb;
SELECT DATABASE();
CREATE TABLE Employee
(
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    DepartmentID INT,
    Salary DECIMAL(10,2)
);
INSERT INTO Employee VALUES
(101,'Rahul',1,30000),
(102,'Sneha',1,50000),
(103,'Amit',2,25000),
(104,'Priya',2,45000),
(105,'Kiran',1,40000);
SELECT DepartmentID,
AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY DepartmentID;
```

Create View

```
CREATE VIEW LowSalaryEmployees AS
SELECT
EmployeeID,
EmployeeName,
DepartmentID,
Salary
FROM Employee e
WHERE Salary <
(
    SELECT AVG(Salary)
    FROM Employee
    WHERE DepartmentID = e.DepartmentID
);
SELECT * FROM LowSalaryEmployees;
```

Update salary by 10%

```
UPDATE Employee e
JOIN LowSalaryEmployees l
ON e.EmployeeID = l.EmployeeID
SET e.Salary = e.Salary * 1.10;
SELECT * FROM Employee;
```

OUTPUT:

```
mysql> SELECT * FROM LowSalaryEmployees;
```

EmployeeID	EmployeeName	DepartmentID	Salary
101	Rahul	1	30000.00
103	Amit	2	25000.00

EmployeeID	EmployeeName	DepartmentID	Salary
101	Rahul	1	30000.00
102	Sneha	1	50000.00
103	Amit	2	25000.00
104	Priya	2	45000.00
105	Kiran	1	40000.00

6. BANKING SYSTEM

QUERY:

CREATE DATABASE bankdb;

USE bankdb;

CREATE TABLE Customer

(
 CustomerID INT PRIMARY KEY,
 CustomerName VARCHAR(50),
 Phone VARCHAR(15)

);

CREATE TABLE Branch

(
 BranchID INT PRIMARY KEY,
 BranchName VARCHAR(50),
 City VARCHAR(50)

);

CREATE TABLE Account

(
 AccountID INT PRIMARY KEY,
 CustomerID INT,
 BranchID INT,
 Balance DECIMAL(10,2),
 FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
 FOREIGN KEY(BranchID) REFERENCES Branch(BranchID)

);

CREATE TABLE TransactionDetails

(
 TransactionID INT PRIMARY KEY,
 AccountID INT,
 Amount DECIMAL(10,2),
 TransactionDate DATE,
 FOREIGN KEY(AccountID) REFERENCES Account(AccountID)

);

INSERT INTO Customer VALUES

(101,'Rahul','9876543210'),

(102,'Sneha','9876543211'),

(103,'Amit','9876543212');

SELECT * FROM Customer;

INSERT INTO Branch VALUES

```

(1,'Chennai Branch','Chennai'),
(2,'Bangalore Branch','Bangalore');
SELECT * FROM Branch;
INSERT INTO Account VALUES
(1001,101,1,50000),
(1002,102,2,75000),
(1003,103,1,60000);
SELECT * FROM Account;
INSERT INTO TransactionDetails VALUES
(1,1001,5000,'2026-08-01'),
(2,1001,3000,'2026-08-02'),
(3,1002,7000,'2026-08-03'),
(4,1003,2000,'2026-08-04');
SELECT * FROM TransactionDetails;
SELECT
    c.CustomerName,
    a.AccountID,
    a.Balance,
    t.TransactionID,
    t.Amount,
    t.TransactionDate,
    b.BranchName,
    b.City
FROM Customer c
JOIN Account a
ON c.CustomerID = a.CustomerID
JOIN TransactionDetails t
ON a.AccountID = t.AccountID
JOIN Branch b
ON a.BranchID = b.BranchID;

```

OUTPUT:

CustomerName	AccountID	Balance	TransactionID	Amount	TransactionDate	BranchName	City
Rahul	1001	50000.00	1	5000.00	2026-08-01	Chennai Branch	Chennai
Rahul	1001	50000.00	2	3000.00	2026-08-02	Chennai Branch	Chennai
Amit	1003	60000.00	4	2000.00	2026-08-04	Chennai Branch	Chennai
Sneha	1002	75000.00	3	7000.00	2026-08-03	Bangalore Branch	Bangalore

7. RETAIL STORE

SQL QUERY:

```

CREATE DATABASE retaildb;
USE retaildb;
CREATE TABLE Sales
(
    SaleID INT PRIMARY KEY,
    Category VARCHAR(50),
    ProductName VARCHAR(50),

```

```

    Amount DECIMAL(10,2),
    SaleDate DATE
);
INSERT INTO Sales VALUES
(1,'Electronics','Laptop',55000,'2026-08-01'),
(2,'Electronics','Mouse',1200,'2026-08-01'),
(3,'Furniture','Chair',6000,'2026-08-01'),
(4,'Furniture','Table',7000,'2026-08-01'),
(5,'Stationery','Notebook',3000,'2026-08-01'),
(6,'Stationery','Pen',1000,'2026-08-01');
SELECT * FROM Sales;
SELECT
    Category,
    SUM(Amount) AS TotalSales
FROM Sales
GROUP BY Category
HAVING SUM(Amount) > 10000;

```

OUTPUT:

```
mysql> SELECT * FROM Sales;
```

SaleID	Category	ProductName	Amount	SaleDate
1	Electronics	Laptop	55000.00	2026-08-01
2	Electronics	Mouse	1200.00	2026-08-01
3	Furniture	Chair	6000.00	2026-08-01
4	Furniture	Table	7000.00	2026-08-01
5	Stationery	Notebook	3000.00	2026-08-01
6	Stationery	Pen	1000.00	2026-08-01

Category	TotalSales
Electronics	56200.00
Furniture	13000.00

Q8. LIBRARY SYSTEM

Tuple Relational Calculus:

```

{ M |
Member(M)
^
∀G (Genre(G) →
∃B∃BR
(Book(B) ^
B.Genre = G.Genre
^ Borrow(BR)
^ BR.MemberID = M.MemberID
^ BR.BookID = B.BookID))
}

```

QUERY:

```
mysql> CREATE DATABASE librarydb;
Query OK, 1 row affected (0.02 sec)

mysql> USE librarydb;
Database changed
mysql> CREATE TABLE Member
-> (
-> MemberID INT PRIMARY KEY,
-> MemberName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Book
-> (
-> BookID INT PRIMARY KEY,
-> BookName VARCHAR(50),
-> Genre VARCHAR(30)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Borrow
-> (
-> BorrowID INT PRIMARY KEY,
-> MemberID INT,
-> BookID INT,
-> BorrowDate DATE,
-> FOREIGN KEY(MemberID) REFERENCES Member(MemberID),
-> FOREIGN KEY(BookID) REFERENCES Book(BookID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Member VALUES
-> (1,'Rahul'),
-> (2,'Sneha'),
-> (3,'Amit');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Member;
+-----+-----+
| MemberID | MemberName |
+-----+-----+
| 1 | Rahul |
| 2 | Sneha |
| 3 | Amit |
+-----+-----+
3 rows in set (0.00 sec)

mysql> INSERT INTO Book VALUES
-> (101,'DBMS','Education'),
-> (102,'Harry Potter','Fantasy'),
-> (103,'Physics','Science'),
-> (104,'C Programming','Education'),
-> (105,'The Hobbit','Fantasy'),
-> (106,'Chemistry','Science');
Query OK, 6 rows affected (0.02 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Book;
+-----+-----+-----+
| BookID | BookName | Genre |
+-----+-----+-----+
| 101 | DBMS | Education |
| 102 | Harry Potter | Fantasy |
| 103 | Physics | Science |
| 104 | C Programming | Education |
| 105 | The Hobbit | Fantasy |
| 106 | Chemistry | Science |
+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> INSERT INTO Borrow VALUES
-> (1,1,101,'2026-08-01'),
-> (2,1,102,'2026-08-02'),
-> (3,1,103,'2026-08-03'),
-> (4,2,101,'2026-08-04'),
-> (5,2,102,'2026-08-05'),
-> (6,3,103,'2026-08-06');
```

```
mysql> SELECT * FROM Borrow;
+-----+-----+-----+-----+
| BorrowID | MemberID | BookID | BorrowDate |
+-----+-----+-----+-----+
| 1 | 1 | 101 | 2026-08-01 |
| 2 | 1 | 102 | 2026-08-02 |
| 3 | 1 | 103 | 2026-08-03 |
| 4 | 2 | 101 | 2026-08-04 |
| 5 | 2 | 102 | 2026-08-05 |
| 6 | 3 | 103 | 2026-08-06 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> SELECT
-> m.MemberID,
-> m.MemberName
-> FROM Member m
-> JOIN Borrow br ON m.MemberID = br.MemberID
-> JOIN Book b ON br.BookID = b.BookID
-> GROUP BY m.MemberID, m.MemberName
-> HAVING COUNT(DISTINCT b.Genre) =
-> (
-> SELECT COUNT(DISTINCT Genre)
-> FROM Book
-> );
```

OUTPUT:

```
+-----+-----+
| MemberID | MemberName |
+-----+-----+
| 1 | Rahul |
+-----+-----+
```

9. INVENTORY MANAGEMENT SYSTEM

```
mysql> USE inventorydb;
Database changed
mysql> CREATE TABLE Supplier
-> (
-> SupplierID INT PRIMARY KEY,
-> SupplierName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Product
-> (
-> ProductID INT PRIMARY KEY,
-> ProductName VARCHAR(50),
-> Price DECIMAL(10,2),
-> CHECK (Price > 0)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Inventory
-> (
-> InventoryID INT PRIMARY KEY,
-> ProductID INT,
-> SupplierID INT,
-> Quantity INT CHECK (Quantity >= 0),
-> FOREIGN KEY(ProductID)
-> REFERENCES Product(ProductID),
-> FOREIGN KEY(SupplierID)
-> REFERENCES Supplier(SupplierID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Supplier VALUES
-> (1,'ABC Suppliers'),
-> (2,'XYZ Traders');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Supplier;
+-----+-----+
| SupplierID | SupplierName |
+-----+-----+
| 1 | ABC Suppliers |
| 2 | XYZ Traders |
+-----+-----+
2 rows in set (0.00 sec)

mysql> INSERT INTO Product VALUES
-> (101,'Laptop',55000),
-> (102,'Mouse',500),
-> (103,'Keyboard',1200);
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProductID | ProductName | Price |
+-----+-----+-----+
| 101 | Laptop | 55000.00 |
| 102 | Mouse | 500.00 |
| 103 | Keyboard | 1200.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> INSERT INTO Inventory VALUES
-> (1,101,1,20),
-> (2,102,2,50),
-> (3,103,1,30);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Inventory;
+-----+-----+-----+-----+
| InventoryID | ProductID | SupplierID | Quantity |
+-----+-----+-----+-----+
| 1 | 101 | 1 | 20 |
| 2 | 102 | 2 | 50 |
| 3 | 103 | 1 | 30 |
+-----+-----+-----+-----+
```

```
mysql> INSERT INTO Product VALUES
-> (104, 'Printer', -5000);
ERROR 3819 (HY000): Check constraint 'product_chk_1' is violated.
mysql> INSERT INTO Inventory VALUES
-> (4, 999, 1, 10);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('inventorydb'. 'inventory', CONSTRAINT 'inventory_ibfk_1' FOREIGN KEY (
'ProductID') REFERENCES 'product' ('ProductID'))
mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProductID | ProductName | Price |
+-----+-----+-----+
| 101 | Laptop | 55000.00 |
| 102 | Mouse | 500.00 |
| 103 | Keyboard | 1200.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Inventory;
+-----+-----+-----+-----+
| InventoryID | ProductID | SupplierID | Quantity |
+-----+-----+-----+-----+
| 1 | 101 | 1 | 20 |
| 2 | 102 | 2 | 50 |
| 3 | 103 | 1 | 30 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

10. HR SYSTEM

Relational Schema

Employee(EmployeeID, EmployeeName, DepartmentID)

Attendance(EmployeeID, AttendanceDate, Status)

Performance(EmployeeID, Month, Rating)

QUERY:

```
mysql> CREATE DATABASE hrdb;
Query OK, 1 row affected (0.02 sec)

mysql> USE hrdb;
Database changed
mysql> CREATE TABLE Employee
-> (
-> EmployeeID INT PRIMARY KEY,
-> EmployeeName VARCHAR(50),
-> Department VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Attendance
-> (
-> AttendanceID INT PRIMARY KEY,
-> EmployeeID INT,
-> AttendanceDate DATE,
-> Status VARCHAR(10),
-> FOREIGN KEY(EmployeeID)
-> REFERENCES Employee(EmployeeID)
-> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO Attendance VALUES
-> (1, 101, '2026-08-01', 'Present'),
-> (2, 101, '2026-08-02', 'Present'),
-> (3, 102, '2026-08-01', 'Absent'),
-> (4, 102, '2026-08-02', 'Present'),
-> (5, 103, '2026-08-01', 'Present'),
-> (6, 103, '2026-08-02', 'Present');
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Attendance;
+-----+-----+-----+-----+
| AttendanceID | EmployeeID | AttendanceDate | Status |
+-----+-----+-----+-----+
| 1 | 101 | 2026-08-01 | Present |
| 2 | 101 | 2026-08-02 | Present |
| 3 | 102 | 2026-08-01 | Absent |
| 4 | 102 | 2026-08-02 | Present |
| 5 | 103 | 2026-08-01 | Present |
| 6 | 103 | 2026-08-02 | Present |
+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

```
mysql>
mysql> INSERT INTO Performance VALUES
-> (1, 101, 5, 'August'),
-> (2, 102, 4, 'August'),
-> (3, 103, 5, 'August');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Performance;
+-----+-----+-----+-----+
| PerformanceID | EmployeeID | Rating | ReviewMonth |
+-----+-----+-----+-----+
| 1 | 101 | 5 | August |
| 2 | 102 | 4 | August |
| 3 | 103 | 5 | August |
+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

```
mysql> CREATE TABLE Performance
-> (
-> PerformanceID INT PRIMARY KEY,
-> EmployeeID INT,
-> Rating INT,
-> ReviewMonth VARCHAR(20),
-> FOREIGN KEY(EmployeeID)
-> REFERENCES Employee(EmployeeID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Employee VALUES
-> (101, 'Rahul', 'IT'),
-> (102, 'Sneha', 'HR'),
-> (103, 'Amit', 'Finance');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Employee;
+-----+-----+-----+
| EmployeeID | EmployeeName | Department |
+-----+-----+-----+
| 101 | Rahul | IT |
| 102 | Sneha | HR |
| 103 | Amit | Finance |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> INSERT INTO Attendance VALUES
```

OUTPUT:

Create Attendance Summary View

```
mysql> CREATE VIEW MonthlyAttendanceSummary AS
-> SELECT
-> e.EmployeeID,
-> e.EmployeeName,
-> COUNT(a.AttendanceID) AS TotalAttendance
-> FROM Employee e
-> JOIN Attendance a
-> ON e.EmployeeID=a.EmployeeID
-> WHERE a.Status='Present'
-> GROUP BY e.EmployeeID,e.EmployeeName;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> SELECT * FROM MonthlyAttendanceSummary;
+-----+-----+-----+
| EmployeeID | EmployeeName | TotalAttendance |
+-----+-----+-----+
| 101 | Rahul | 2 |
| 102 | Sneha | 1 |
| 103 | Amit | 2 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Create Performance Summary View

```
mysql> CREATE VIEW MonthlyPerformanceSummary AS
-> SELECT
-> e.EmployeeID,
-> e.EmployeeName,
-> AVG(p.Rating) AS AverageRating
-> FROM Employee e
-> JOIN Performance p
-> ON e.EmployeeID=p.EmployeeID
-> GROUP BY e.EmployeeID,e.EmployeeName;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> SELECT * FROM MonthlyPerformanceSummary;
+-----+-----+-----+
| EmployeeID | EmployeeName | AverageRating |
+-----+-----+-----+
| 101 | Rahul | 5.0000 |
| 102 | Sneha | 4.0000 |
| 103 | Amit | 5.0000 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM MonthlyPerformanceSummary;
```